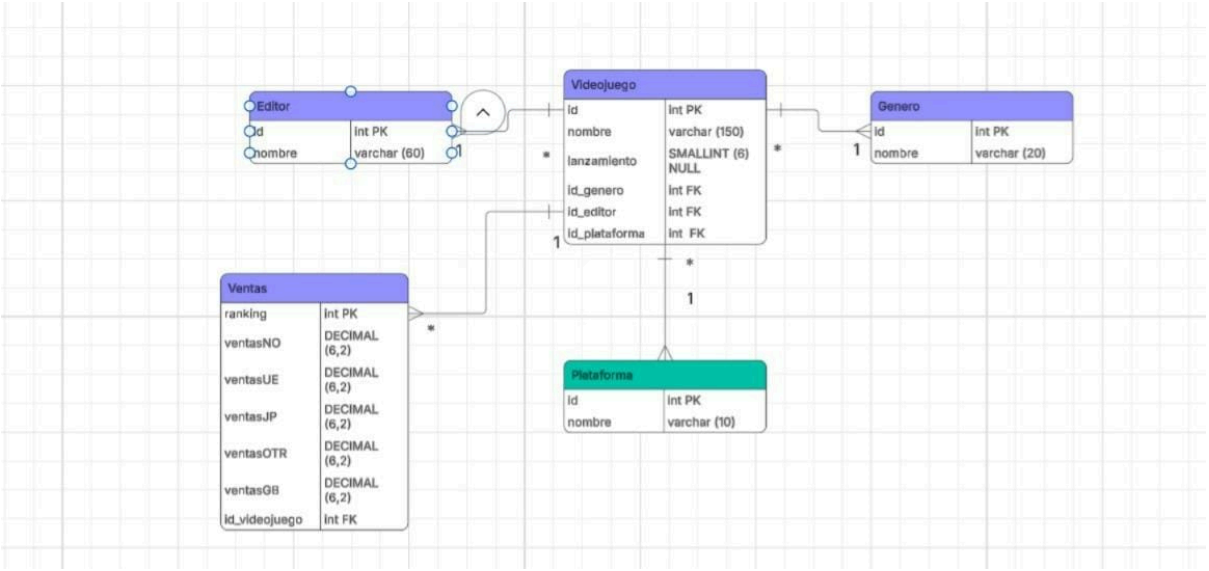
Geronimo Visconti, Martin Fogar
Tecnicatura Superior en Desarrollo de Software, Escuela Superior de Comercio N° 43
3°, Gestión de Proyectos de Software-Base de Datos
Nardelli Ariel, Nardelli Matias
2026



El dataset elegido por el equipo corresponde a un conjunto de datos de ventas de videojuegos, clasificados mediante un ranking global y regional. El conjunto contiene información sobre los videojuegos, sus plataformas, años de lanzamiento, géneros, editores y ventas registradas en distintas regiones, además del total de ventas globales.
Los datos originales son:

Campo	Descripción	Tema
Rank	Ranking global de ventas	Entero
Name	Nombre del videojuego	Texto
Platform	Plataforma	Texto
Year	Año de lanzamiento	Entero / NULL
Genre	Género	Texto
Publisher	Editor	Texto
NA_Sales	Ventas de Norteamérica	Decimal
EU_Sales	Ventas en Europa	Decimal
JP_Sales	Ventas en Japón	Decimal
Other_Sales	Ventas en otras regiones	Decimal
Global_Sales	Ventas globales	Decimal

El diagrama entidad relacion quedo de la siguiente manera:



La descripción y justificación de cada una de las tablas:

Videojuegos: Contiene la información principal de cada registro de los videojuegos. Los videojuegos se pueden repetir por nombre, pero se consideran diferentes según la plataforma en la cual fueron lanzados.

- id: Clave primaria.
- nombre: Nombre del videojuego.
- lanzamiento: Año del lanzamiento del videojuego.
- id_genero: Clave foránea, Género del videojuego.
- id_editor: Clave foránea, Editor del videojuego.

Videojuegos	
id	int PK
nombre	varchar (150)
lanzamiento	SMALLINT (6) NULL
id_genero	int FK
id_editor	int FK
id_plataforma	int FK

Plataforma: Contiene las diferentes plataformas en donde fueron lanzados los videojuegos.

- id: Clave primaria.
- nombre: Nombre de la plataforma.

Plataforma	
id	int PK
nombre	varchar (10)

Género: Contiene los generos de los videojuegos.

- id: Clave primaria.
- nombre: Nombre del género del videojuego.

Genero	
id	int PK
nombre	varchar (20)

Editor: Contiene los editores que publicaron los videojuegos.

- id: Clave primaria.
- nombre: Nombre del editor.

Editor	
id	int PK
nombre	varchar (60)

Ventas: Contiene el ranking y el registro de ventas de los videojuegos, diferenciándolos por regiones y su total de ventas a nivel global.

- ranking: Clave primaria, ranking de ventas totales del videojuego.
- ventasNO: Ventas del videojuego en Norteamérica (en millones).

- ventasUE: Ventas del videojuego en Europa (en millones).
- ventasJP: Ventas del videojuego en Japón (en millones).
- ventasOTR: Ventas del videojuego en el resto del mundo(en millones).
- ventasGB: Ventas totales a nivel mundial (en millones).
- id_videojuego: Clave Foránea, id del videojuego.

Ventas	
ranking	int PK
VentasNO	DECIMAL (6,2)
VentasUE	DECIMAL (6,2)
VentasJP	DECIMAL (6,2)
VentasOTR	DECIMAL (6,2)
VentasGB	DECIMAL (6,2)
id_videojuego	int FK

ranking (máximo 5 caracteres)
nombre (máximo 132 caracteres)
plataforma (máximo 4 caracteres)
lanzamiento (máximo 4 caracteres)
genero (máximo 12 caracteres)
editor (máximo 38 caracteres)
ventas (máximo 5 caracteres)

Importación de datos:

Para cargar el archivo CSV creamos una base de datos provisoria, usada solo como paso intermedio. En esta se creó una tabla ventas con las 11 columnas del CSV, todas de tipo VARCHAR y sin restricciones, para que recibiera los datos tal cual vienen en el archivo.

```
CREATE TABLE ventas(
 RANK VARCHAR(100),
 NAME VARCHAR(150),
 Platform VARCHAR(150),
 YEAR VARCHAR(150),
 Genre VARCHAR(150),
 Publisher VARCHAR(150),
 NA_Sales VARCHAR(150),
 EU_Sales VARCHAR(150),
 JP_Sales VARCHAR(150),
 Other_Sales VARCHAR(150),
 Global_Sales VARCHAR(150));
```

El archivo CSV se importa en esa tabla con HeidiSQL (Importar archivo de texto), con los campos terminados por coma, delimitados por comillas dobles, líneas terminadas por \n y omitiendo la primera línea (el encabezado). El método de importación utilizado fue “El servidor analiza el contenido del archivo (LOAD DATA)”.

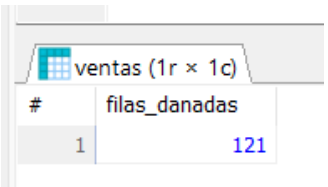
Cantidad de registros cargados

El dataset original tiene 16.598 registros, pero al importar no carga el total real. Queda con 16.475 filas: 16.354 correctas y 121 dañadas.

El motivo es que algunos campos del CSV contienen comas dentro de su contenido, por ejemplo el nombre “Hey You, Pikachu!”, y por eso van entre comillas. Con el método utilizado, al importar y no se interpreta esas comillas, lo cual no puede determinar dónde termina cada fila. Entonces la línea completa del CSV queda guardada dentro de la columna RANK, las columnas siguientes quedaron desplazadas con datos de otro videojuego, y la línea siguiente del archivo fue absorbida por la misma fila. Por eso el total de filas es menor al del archivo: 121 filas dañadas contienen 244 líneas del CSV.

Las filas dañadas se identificaron porque el valor de RANK no era un número:

```
SELECT COUNT(*) FROM ventas WHERE ventas.RANK NOT REGEXP ‘^[0-9]+$’;
```



ventas (1r × 1c)

#	filas_danadas
1	121

DELETE FROM ventas WHERE ventas.RANK NOT REGEXP '^[0-9]+$';

Una vez borrados esos registros, la base final cuenta con 16.354, 244 menos que el dataset original.

Creación de la base de datos:

Como primer paso se crea la base de datos que almacenará toda la información correspondiente al dataset de videojuegos. Para ello, se utiliza el siguiente código SQL:

```
CREATE DATABASE practicobd;
USE practicobd;
```

Creación de las tablas

Una vez creada y seleccionada la base de datos, se procede a crear las tablas que conforman el modelo relacional. Las tablas se crean respetando las relaciones establecidas en el modelo entidad-relación.

Tabla plataforma:

La tabla plataforma almacena las diferentes plataformas en las que fueron publicados los videojuegos.

```
CREATE TABLE plataforma (
   id INT AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(10) NOT NULL
);
```

El campo id funciona como clave primaria y se genera automáticamente mediante AUTO_INCREMENT. Esto permite asignar un identificador único a cada plataforma, aunque dicho identificador no se encuentre presente en el dataset original, esta parte corresponde a cada tabla ya que ninguna tenía de por sí un identificador único como id. El campo nombre almacena el nombre de la plataforma y no permite valores nulos.

Tabla genero:

La tabla genero contiene los distintos géneros de videojuegos presentes en el dataset.

```
CREATE TABLE genero (
   id INT AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(20) NOT NULL);
```

Al igual que en la tabla plataforma, se utiliza un identificador numérico autoincremental como clave primaria. El campo nombre almacena la descripción del género.

Tabla editor:

La tabla editor almacena los nombres de las empresas editoras o publishers de los videojuegos.

```
CREATE TABLE editor (
   id INT AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(60) NOT NULL
);
```

El campo id identifica de manera única a cada editor y se genera automáticamente. El campo nombre contiene el nombre de la empresa editora.

Tabla videojuegos:

Contiene la información propia de cada videojuego y las referencias hacia su género, editor y plataforma.

```
CREATE TABLE videojuegos (
   id INT AUTO_INCREMENT PRIMARY KEY,
   nombre VARCHAR(150) NOT NULL,
   lanzamiento SMALLINT(6) NULL,
   id_genero INT NOT NULL,
   FOREIGN KEY (id_genero) REFERENCES genero(id),
   id_editor INT NOT NULL,
   FOREIGN KEY (id_editor) REFERENCES editor(id),
   id_plataforma INT NOT NULL,
   FOREIGN KEY (id_plataforma) REFERENCES plataforma(id)
);
```

El campo ranking identifica de manera única a cada videojuego. nombre almacena su nombre, lanzamiento contiene el año de lanzamiento. Este último permite valores nulos debido a que en el dataset original existen registros cuyo año de lanzamiento no se encuentra disponible.

Los campos id_genero, id_editor e id_plataforma funcionan como claves foráneas y permiten establecer las relaciones entre la tabla videojuegos y las tablas genero, editor y plataforma.

Tabla ventas:

Finalmente, se crea la tabla ventas, destinada a almacenar los datos de ventas de cada videojuego según las distintas regiones y el total global.

```
CREATE TABLE ventas (
    ranking INT AUTO_INCREMENT PRIMARY KEY,
    ventasNO DECIMAL(6,2) NOT NULL,
    ventasUE DECIMAL(6,2) NOT NULL,
    ventasJP DECIMAL(6,2) NOT NULL,
    ventasOTR DECIMAL(6,2) NOT NULL,
    ventasGB DECIMAL(6,2) NOT NULL,
    id_videojuego INT NOT NULL,
    FOREIGN KEY (id_videojuego) REFERENCES videojuegos(id)
);
```

Se utiliza el tipo DECIMAL(6,2) debido a que los valores de ventas del dataset contienen números decimales.

El campo id_videojuego funciona como clave foránea y establece la relación entre cada registro de ventas y el videojuego correspondiente.

Las tablas se crean en el siguiente orden:

1. plataforma
2. género
3. editor
4. videojuegos
5. ventas

Consultas para pasar los datos de una base de datos a otra:

Carga de plataformas:

```
INSERT INTO practicobd.plataforma(nombre)

SELECT DISTINCT Platform

FROM bdpractico.ventas

WHERE Platform IS NOT NULL;
```

Se utiliza DISTINCT para evitar insertar registros duplicados y WHERE Platform IS NOT NULL para excluir los valores nulos.

Carga de editores:

```
INSERT INTO practicobd.editor(nombre)

SELECT DISTINCT Publisher

FROM bdpractico.ventas

WHERE Publisher IS NOT NULL;
```

Carga de géneros:

```
INSERT INTO practicobd.genero(nombre)

SELECT DISTINCT Genre

FROM bdpractico.ventas

WHERE Genre IS NOT NULL;
```

Eliminación de registros con datos inválidos:

```
DELETE FROM bdpractico.ventas

WHERE `RANK` NOT REGEXP '^[0-9]+$';
```

Estos registros presentan datos dañados o inconsistentes, por lo que se eliminan antes de realizar la carga hacia las tablas de la base de datos principal.

Luego de esta limpieza, la tabla queda con 16.354 registros válidos.

Carga de videojuegos:

```
INSERT INTO practicobd.videojuegos

(id, nombre, lanzamiento, id_genero, id_editor, id_plataforma)

SELECT

    ventas.RANK,

    ventas.Name,

    NULLIF(ventas.Year, 'N/A'),

    genero.id,

    editor.id,

    plataforma.id

FROM bdpractico.ventas AS ventas

JOIN practicobd.genero AS genero

    ON ventas.Genre = genero.nombre

JOIN practicobd.editor AS editor

    ON ventas.Publisher = editor.nombre

JOIN practicobd.plataforma AS plataforma

    ON ventas.Platform = plataforma.nombre;
```

Esta consulta realiza la carga de los videojuegos desde la tabla de origen hacia practicobd.videojuegos.

El campo RANK se utiliza como identificador del videojuego, mientras que Name corresponde al nombre y Year al año de lanzamiento. La función NULLIF convierte el valor 'N/A' en NULL, ya que representa un dato desconocido.

Los JOIN permiten obtener los identificadores correspondientes de género, editor y plataforma a partir de sus nombres, estableciendo así las relaciones entre las tablas mediante sus respectivas claves foráneas.

Limpieza de los datos de ventas:

Importante: antes de realizar la carga de las ventas, es necesario limpiar la columna Global_Sales.

```
UPDATE bdpractico.ventas

SET Global_Sales = REPLACE(

   REPLACE(Global_Sales, CHAR(13), ''),

   ';',

   ''

);
```

Esta consulta se ejecuta porque la columna Global_Sales de bdpractico.ventas contiene caracteres sobrantes al final de los valores: un punto y coma (;) y un retorno de carro (CHAR(13)).

Carga de ventas:

Una vez realizada la limpieza, se pueden insertar los datos en la tabla practicobd.ventas:

```
INSERT INTO practicobd.ventas

(ranking, ventasNO, ventasUE, ventasJP, ventasOTR, ventasGB, id_videojuego)
```

```
SELECT
    ventas.RANK,
    ventas.NA_Sales,
    ventas.EU_Sales,
    ventas.JP_Sales,
    ventas.Other_Sales,
    ventas.Global_Sales,
    ventas.RANK
FROM bdpractico.ventas AS ventas;|
```

Esta consulta transfiere los datos de ventas desde la tabla de origen hacia la tabla practicobd.ventas.

Además, RANK se utiliza para completar tanto el campo ranking como id_videojuego, relacionando de esta manera cada registro de ventas con el videojuego correspondiente.

Vistas:

Estas permiten obtener determinados conjuntos de datos sin necesidad de escribir nuevamente las consultas utilizadas para relacionar y filtrar las tablas.

Vista para consultar videojuegos de la plataforma N64

Esta vista permite obtener los videojuegos asociados a la plataforma N64. Para ello, se filtran los registros de la tabla videojuegos cuyo id_plataforma corresponde al identificador asignado a N64.

```
CREATE VIEW videojuegos_n64 AS
SELECT videojuegos.nombre, videojuegos.id_plataforma
FROM videojuegos
WHERE id_plataforma = 12;
```

Para consultar la información almacenada en la vista:

```
SELECT * FROM videojuegos_n64;
```

Vista para consultar videojuegos del género Misc junto con sus ventas globales

Esta vista permite consultar los videojuegos pertenecientes al género Misc y sus respectivas ventas globales. Para obtener esta información se relacionan las tablas videojuegos y ventas mediante el identificador del videojuego.

```
CREATE VIEW videojuegos_misc AS
SELECT j.nombre, j.id_genero, v.ventasGB
FROM videojuegos j, ventas v
WHERE j.id = v.id_videojuego
  AND j.id_genero = 6;
```

Para consultar la vista:

```
SELECT * FROM videojuegos_misc;
```

Vista para consultar las ventas globales por plataforma

Esta vista permite obtener el total de ventas globales acumuladas para cada plataforma. Para ello, se relacionan las tablas plataforma, videojuegos y ventas, y posteriormente se utiliza la función SUM() para calcular las ventas totales de cada plataforma.

```
CREATE VIEW ventas_globales_plataforma AS
SELECT
    plataforma.nombre AS plataforma,
    SUM(ventas.ventasGB) AS total_ventas
FROM plataforma
JOIN videojuegos
    ON videojuegos.id_plataforma = plataforma.id
JOIN ventas
    ON ventas.id_videojuego = videojuegos.id
GROUP BY plataforma.nombre
ORDER BY total_ventas DESC;
```

La consulta agrupa los resultados por plataforma y ordena las plataformas de acuerdo con sus ventas globales, desde el mayor valor hasta el menor.

Para consultar la vista:

```
SELECT * FROM ventas_globales_plataforma;
```

Vista para consultar los 20 videojuegos con mayores ventas globales

Esta vista permite obtener los 20 videojuegos que presentan los mayores valores de ventas globales, mostrando además la plataforma correspondiente. Los resultados se ordenan de manera descendente según el total de ventas globales.

```
CREATE VIEW top_20_juegos AS
SELECT
   videojuegos.nombre AS juego,
   plataforma.nombre AS plataforma,
   ventas.ventasGB AS ventas_globales
FROM videojuegos
JOIN plataforma
   ON videojuegos.id_plataforma = plataforma.id
JOIN ventas
   ON ventas.id_videojuego = videojuegos.id
ORDER BY ventas.ventasGB DESC
LIMIT 20;
```

En este caso, ventasGB corresponde a las **ventas globales** del videojuego y no al ranking original del dataset. La consulta utiliza ORDER BY ... DESC para ordenar los videojuegos desde el que posee mayores ventas globales hasta el que posee menores ventas, limitando el resultado a los primeros 20 registros.

Para consultar la vista:

```
SELECT * FROM top_20_juegos;
```

Índices

Además de las vistas, se creó un índice para optimizar las consultas relacionadas con el nombre y el año de lanzamiento de los videojuegos.

Índice sobre nombre y año de lanzamiento

El siguiente comando crea un índice denominado videojuegos_lanzamiento sobre los campos nombre y lanzamiento de la tabla videojuegos:

```
CREATE INDEX videojuegos_lanzamiento
ON videojuegos (nombre, lanzamiento);
```

Este índice permite mejorar el acceso a los registros cuando se realizan búsquedas o consultas que utilizan estos campos, facilitando la localización de videojuegos a partir de su nombre y año de lanzamiento.